

Algorithm 1 of DA A

```

IF ((A3-0 > 9) OR (AC = 1)):
{
    A = A + 6      // or AL=AL+06
}
IF ((A7-4 > 9) OR (CY= 1)):
{
    A = A + 60h    // or AH=AH+06
}
ELSE:
Nothing to be performed;

```

DA instruction works only on accumulator A (for example, DA A). This means that the source can be an operand of any addressing mode, but destination must be accumulator. BCD operand can never have any digit greater than 9. So, DA instruction must always be used after addition of BCD operands. DA instruction works after ADD/ADDC instruction in 8051.

To understand the functionality of DA A instruction, consider the following examples. For proper explanation, accumulator is logically divided into two sections—AL and AH—representing the lower and upper nibbles of the accumulator.

Consider example 4 given in Table III. Required output 13d is obtained in decimal form without any error. This conventional algorithm works as follows. After addition process, condition 2 of Algorithm 1 is achieved. So the compiler adds 6 to A or AL register to obtain the result, which is 13d.

In Example 5 given in Table III, required result is 42d after adding the binary for 1Ch and 14d. But, algorithm 1 failed because of the following reasons. After addition process, value of AL is not greater than 9 and AC = 1. Step 3 of algorithm 1 is satisfied and AL register is added with 6. But in this process, decimal value is seen as 36d instead of 42d.

From the above analysis, it can be concluded that DA A instruction in 8051 does not work properly in some situations. Bugs can be easily

TABLE III
EXAMPLES FOR ALGORITHM 1

Example 4	Example 5
09d+04d = 13d	1Ch + 14d = 42d
0000 1001 + 0000 0100 0000 1101 (0C) ===== AC=0, AL>9 CY=0, AH!>9	0001 1100 + 0001 0100 0011 0000 (30d) ===== AC=1, AL!>9 CY=0, AH!>9
0000 1101 + (0C) 0110 0001 0011 (13d)	0011 0000 + 0110 0011 0110 (36d)

TABLE IV
EXAMPLE FOR ALGORITHM 2

Example 6
1Ch + 14d = 42d
0001 1100 + 0001 0100 0011 0000 (30d) ===== AC=1, AL!>9 CY=0, AH!>9
0011 0000 + (30) 0110 0011 0110 + (36) 0110 0011 1100 + (3C) 0110 0100 0010 + (42d)

corrected as shown in Algorithm 2. The algorithm is designed and tested in LabVIEW graphical programming platform.

Novel architecture of DA instruction

This modified approach on DA instruction offers an error-free decimal conversion process efficiently. The procedure for DA A instruction in 8051 MCUs is explained below.

1. Compute the sum of the numbers using 8051.
2. Take the value of AH as counter.
3. Add 6d to AL register as many

times as the counter AH registers.

Consider the instruction for addition operation ADD A, B. Take test values from example 5 for this operation. Register A holds value 1C, and B register holds value 14. After adding 1C and 14, result is 30h.

As per Algorithm 2, load the counter with the value of AH register. In this case, AH is 3, so counter value will be 3. Addition of the number 6 three times (0

to 2) to AL register can be performed. The result obtained is 42. This is the correct expected result after application of Algorithm 2. Detailed steps of this explanations are shown in Example 6 in Table IV.

Algorithm 2

```

IF ((A3-0 > 9) OR (AC = 1)) OR ((A7-4 > 9) OR (CY = 1)):
{
    Counter = AH;
    While (COUNTER! =0):
    {
        AL=AL+6;
        Decrement counter;
    }
}
ELSE:
Nothing to be performed;

```

The above analysis shows that decimal conversion instructions in microprocessors and MCUs have problems. Any 8-bit arithmetic operation like addition and subtraction is sure to have problems.

Algorithm 2 has been tested, and it offers accurate results on all 8-bit MCUs and microprocessors. **EFY**



K.M. Abubeker (L), Reju Elisa Baby and Ajai Mathew are assistant professors at Amal Jyothi College of Engineering, Kanjirapally, Kottayam, Kerala

**To read more stories about
manufacturing and SMT**

Log on to

www.electronicb**2b.com**

